

NanaPay — Cahier de tâches BACKEND

Répartition précise du travail pour 1 développeur Backend. Basé sur le cahier des charges et le schéma SQL NanaPay fournis.

1. Objectif du développeur Backend

Construire l'API, la base de données, l'authentification, les règles métier, les workflows de paiement, les notifications, la gestion des fichiers et les intégrations externes de NanaPay.

- Le Backend est la source de vérité pour les montants, statuts, délais, commissions, autorisations et transitions de commandes.
- Chaque opération financière sensible doit être authentifiée, autorisée, journalisée et idempotente.
- Le Backend doit empêcher qu'un utilisateur modifie directement un montant, un statut ou un bénéficiaire.

2. Base de données à implémenter

Reprendre le schéma du document : users, clients, merchants, shops, products, product_media, orders, savings_plans, savings_deposits, bank_credits, merchant_payouts et order_deliveries.

Ajouter, si nécessaire, des tables techniques pour audit_logs, notifications, refresh_tokens/sessions, fichiers et idempotency_keys, sans supprimer les tables fonctionnelles du schéma.

Mettre en place clés étrangères, contraintes, index, timestamps et transactions SQL.

Les identifiants doivent être uniques; les références de transactions Mobile Money doivent être uniques.

3. Authentification & autorisation

Inscription des quatre rôles : ADMIN, CLIENT, COMMERCEANT, BANQUE.

Hash sécurisé des mots de passe; aucune conservation en clair.

Connexion, session/token, déconnexion et récupération de compte.

RBAC strict : chaque endpoint vérifie le rôle et la propriété de la ressource.

Compte client/commerçant inactif tant que l'Admin n'a pas validé.

4. API CLIENT

POST /auth/register/client : créer le dossier Client.

POST /auth/login : connexion.

GET /client/profile : profil.

GET /market/shops?city=&township;=&search;= : recherche boutiques.

GET /shops/:id/products : articles et stock visible.

POST /orders : créer une commande avec produit et mode Épargne/Crédit.

GET /orders/:id : détail et statut.

POST /orders/:id/savings/extend : demander l'extension si autorisée.

POST /orders/:id/bank-credit : soumettre/réutiliser le dossier crédit.

POST /orders/:id/bank-credit/terms : enregistrer les modalités autorisées.

GET /orders/:id/qr : fournir le QR uniquement si le financement est validé.

GET /client/savings : progression de l'épargne.

GET /client/notifications : notifications.

5. Règles métier ÉPARGNE

0–50 000 FCFA : minimum 1 000 FCFA par versement, maximum 8 mois.

50 000–100 000 FCFA : minimum 2 000 FCFA, maximum 18 mois.

>100 000 FCFA : minimum 5 000 FCFA, maximum 36 mois.

Autoriser une extension maximale de 2 mois avant l'échéance.

Valider automatiquement les dépôts via webhook Mobile Money lorsque l'intégration est disponible.

Mettre à jour `current_saved_amount` de manière transactionnelle.

À 100 %, passer la commande à `PRET_A_LIVRER` et préparer le retrait.

Si le délai définitif est dépassé, appliquer la règle documentée de pénalité : 15 % et remboursement de 85 %, puis blocage temporaire. Le document initial contient une ancienne règle de 30 %/70 %; le schéma/architecture détaillé retient 15 %/85 %, qui doit être utilisé comme règle actuelle.

6. Règles métier CRÉDIT BANCAIRE

Le client soumet identité, employeur, autorisation de prélèvement signée et bulletins de salaire.

Après validation bancaire, l'autorisation déjà validée peut être réutilisée pour les futurs achats.

0–50 000 FCFA : maximum 8 mois.

50 000–100 000 FCFA : maximum 12 mois.

Au-delà, ajouter 6 mois par tranche de 50 000 FCFA selon la règle du document, avec arrondissement supérieur.

Le Backend calcule lui-même la durée et refuse toute durée hors limite.

Créer/mettre à jour `bank_credits` avec approbation, montant de commission 2 %, montant total à virer et statut du virement.

7. API COMMERÇANT

POST/PUT /merchant/shop : gérer la boutique.

POST /merchant/products : créer un article.

PUT /merchant/products/:id : modifier article, prix ou stock.

POST /merchant/products/:id/media : gérer photos/spots.

GET /merchant/orders?status= : ventes en cours et commandes prêtes.

POST /merchant/orders/:id/scan-qr : valider le retrait.

GET /merchant/payouts : historique des gains.

8. Gestion du stock et commande

Lorsqu'un achat réserve un article, décrémenter le stock de façon transactionnelle pour éviter les ventes concurrentes.

Quand `remaining_stock` atteint le `quota/stock` indisponible, `is_visible` doit empêcher les nouvelles sélections.

Ne jamais faire confiance à un prix envoyé par le Front-end : reprendre le prix stocké en base.

Associer chaque commande à un client et un produit valides.

Une commande ne peut devenir LIVRE qu'après validation d'un QR valide par le commerçant autorisé.

9. API BANQUE

GET /bank/credit-requests : dossiers à examiner.

GET /bank/credit-requests/:id : détail et documents.

POST /bank/credit-requests/:id/approve : approuver.

POST /bank/credit-requests/:id/reject : rejeter.

POST /bank/transfers/:id/execute : exécuter/enregistrer le virement vers NanaPay.

Après virement, notifier l'Admin avec client, montant, commerçant et numéro Coris Money.

10. API ADMIN

GET /admin/clients/pending et POST /admin/clients/:id/approve ou reject.

GET /admin/merchants/pending et POST /admin/merchants/:id/approve ou reject.

GET /admin/savings : suivi des épargnes.

GET /admin/credits : suivi des crédits.

POST /admin/credits/:id/payout : procéder au virement commerçant.

GET /admin/transactions et /admin/notifications.

Après validation d'un compte, générer/envoyer automatiquement les identifiants par e-mail selon le mécanisme de messagerie choisi.

11. QR et workflow de livraison

Lorsque Épargne = 100 % OU Crédit bancaire = approuvé + virement reçu, générer un token QR unique, difficile à deviner et à usage contrôlé.

Le QR est affiché au Client et la commande passe à PRET_A_LIVRER.

Le commerçant scanne le QR; le Backend vérifie commande, produit, commerçant autorisé, statut et validité du token.

Si valide : créer order_deliveries, passer la commande à LIVRE et déclencher le workflow de reversement.

Refuser les doubles scans et les scans d'un commerçant non autorisé.

12. Calculs financiers

Virement banque vers NanaPay : $\text{Prix article} + (\text{Prix article} \times 2 / 100)$.

Épargne : le document final indique HT de 8 % pour les articles <50 000 FCFA, 5 % pour 50 000–100 000 FCFA et 3 % pour les produits ≥100 000 FCFA.

Reversement Épargne : montant épargné - commission correspondant au taux HT.

Crédit : commission commerçant de 4 % selon le document; la partie banque issue de l'intérêt doit être gérée séparément.

Le Backend doit centraliser ces règles dans un service financier testable et éviter les nombres « magiques » dans les contrôleurs.

Exemple fourni dans le document : pour un article à 50 000 FCFA, financement bancaire de 55 000 FCFA, virement de 51 000 FCFA à NanaPay, 4 % = 2 000 FCFA côté commerçant et 1 000 FCFA côté banque, puis 48 000 FCFA au commerçant; le document indique un gain d'opération de 2 520 FCFA.

13. Mobile Money, e-mail et fichiers

Créer une abstraction de paiement permettant d'intégrer Orange Money/Coris Money sans mélanger la logique métier avec le fournisseur.

Recevoir et vérifier les webhooks; protéger contre les doublons avec transaction_reference/idempotency key.

Uploader et stocker les CNIB, bulletins et autorisations avec accès contrôlé.

Envoyer les e-mails de validation/identifiants et notifications métier.

Ne jamais mettre les clés API des opérateurs dans le Front-end.

14. Sécurité & audit

Validation stricte de toutes les entrées.

Rate limiting sur authentification et endpoints sensibles.

Contrôle MIME/taille des fichiers et protection contre les uploads dangereux.

Journaliser approbations, rejets, dépôts, virements, scans QR et changements de statut.

Idempotence obligatoire pour webhooks, virements et actions pouvant être répétées.

Transactions DB pour toute opération modifiant argent + statut + stock.

15. Tests Backend obligatoires

- Tests unitaires des règles de durée Épargne/Crédit.
- Tests des commissions et montants de virement.
- Tests de validation des rôles et permissions.
- Tests de stock concurrent.
- Tests webhook avec événement dupliqué.
- Tests de double scan QR.
- Tests de transition de commande.
- Tests validation/rejet Admin et Banque.
- Tests de non-modification du prix depuis le client.
- Tests d'échec d'un paiement ou d'un virement sans corruption des données.

16. Livrable attendu

Une API documentée (OpenAPI/Swagger recommandé), une base SQL migrée, authentification/RBAC, logique métier testée, intégrations externes isolées, journal d'audit et endpoints prêts à être consommés par le Front-end.

Point de coordination obligatoire : le Backend fournit au Front-end un document API avec endpoint, méthode, paramètres, exemple de requête, exemple de réponse, codes d'erreur et droits requis. Le Front-end ne doit pas inventer les contrats API.